

MANUAL TÉCNICO

Gestor Académico Prácticas 2026

Sistema web para la gestión de cursos, notas y comunicación entre estudiantes y docentes

Carlos Roberto Franco Juárez – 201800559
Zabdi Jefe Rivas Martínez — 202300349

Prácticas Iniciales, sección C — Universidad de San Carlos de Guatemala

Introducción

Este manual describe cómo está construido el sistema Gestor. La idea es que cualquier persona que necesite dar mantenimiento al proyecto, o retomarlo más adelante, pueda entender rápidamente cómo se comunican las piezas entre sí: el frontend, el backend y la base de datos.

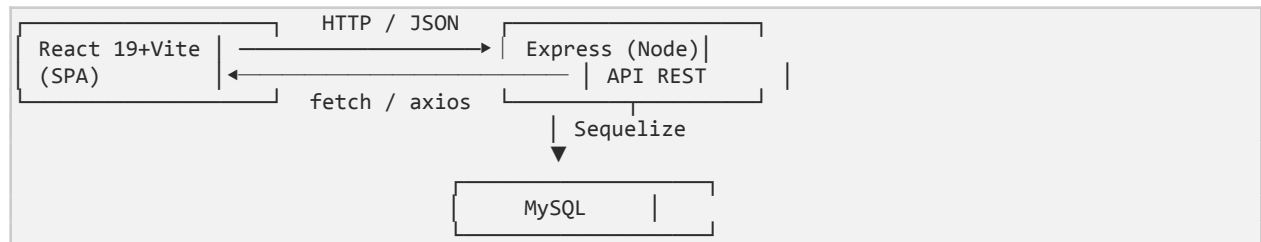
El sistema se organiza en tres capas separadas:

- Un frontend hecho en React 19 con Vite, que es lo que el usuario ve y con lo que interactúa en el navegador.
- Un backend en Node.js con Express, que expone una API REST y contiene toda la lógica de negocio.
- Una base de datos MySQL, a la que el backend accede a través de Sequelize (un ORM que traduce nuestras clases de JavaScript en tablas y consultas SQL).

Arquitectura del sistema

La arquitectura sigue un patrón clásico de aplicación web de tres capas, donde cada capa solo se comunica con la que tiene inmediatamente al lado. El frontend nunca toca la base de datos directamente; siempre pasa por la API.

Diagrama general



En palabras simples: el usuario interactúa con la interfaz de React (una SPA, o "Single Page Application", porque no recarga la página completa en cada acción). Esa interfaz hace peticiones HTTP al backend, normalmente con fetch o axios, dirigidas a rutas como `/api/auth/login` o `/api/courses`. Express recibe esas peticiones, las valida, ejecuta la lógica correspondiente y, cuando necesita leer o escribir datos, usa Sequelize para hablar con MySQL sin tener que escribir SQL a mano en la mayoría de los casos.

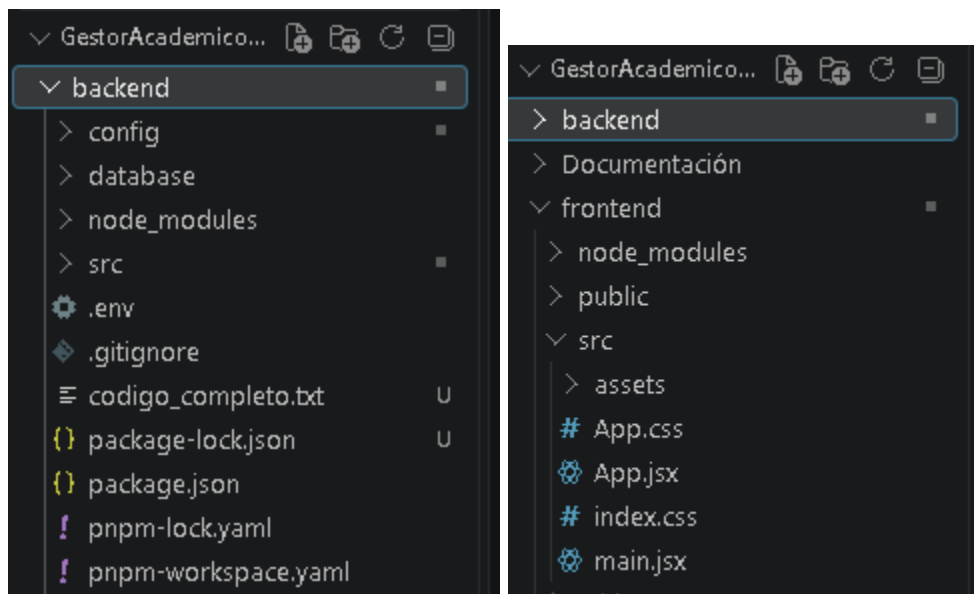
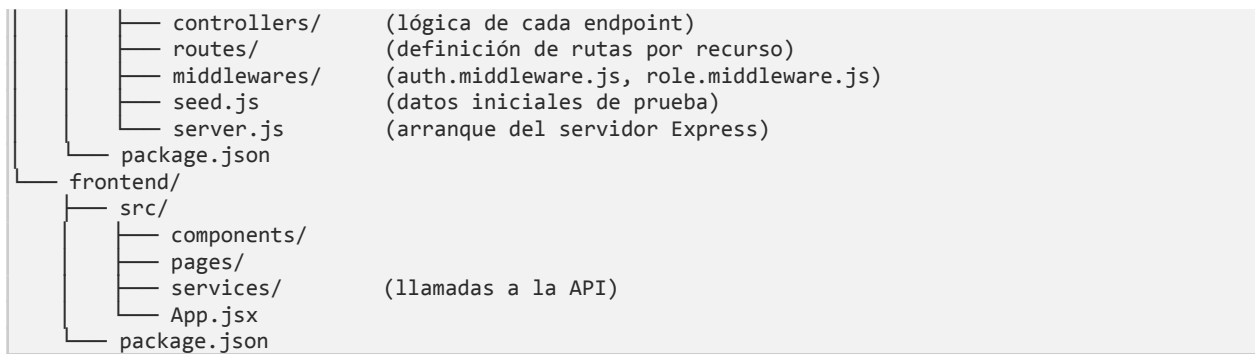
Flujo de una petición típica

1. El usuario hace una acción en la interfaz (por ejemplo, entra a la vista de notas de un curso).
2. El componente de React dispara una petición HTTP (por ejemplo, `GET /api/grades?courseId=3`) e incluye el token JWT en la cabecera `Authorization`.
3. Express recibe la petición y la pasa primero por los middlewares: uno valida que el token sea válido y no haya expirado, y otro valida que el rol del usuario tenga permiso para esa ruta.
4. Si todo pasa, la petición llega al controlador correspondiente, que usa el modelo de Sequelize (por ejemplo `Grade`) para consultar la base de datos.
5. El controlador devuelve una respuesta en JSON, que React recibe y usa para actualizar el estado del componente y, con eso, lo que se ve en pantalla.

Organización de carpetas

Para que el proyecto sea fácil de navegar, conviene mantener una estructura de carpetas ordenada por responsabilidad, separando siempre el backend del frontend en la raíz del repositorio:

```
gestor-academico-practicas-2026/
├── backend/
│   ├── src/
│   │   ├── config/      (conexión a MySQL, variables de entorno)
│   │   └── models/      (User, Course, Grade, Post)
```



Endpoints de la API REST

Todos los endpoints están montados bajo el prefijo /api. A continuación se detalla cada grupo de rutas, su método HTTP, qué recibe, qué devuelve y qué rol necesita quien la consuma. Todas las rutas, salvo el login y el registro, requieren un token JWT válido en la cabecera Authorization: Bearer <token>.

/api/auth — Autenticación

Método	Ruta	Descripción	Acceso
POST	/api/auth/register	Crea un nuevo usuario (nombre, correo, contraseña, rol). La contraseña se guarda con hash (bcrypt).	Público
POST	/api/auth/login	Verifica correo y contraseña; si son correctos, devuelve un token JWT y los datos básicos del usuario.	Público
GET	/api/auth/me	Devuelve los datos del usuario autenticado, a partir del token enviado.	Autenticado

/api/courses — Cursos

Método	Ruta	Descripción	Acceso
GET	/api/courses	Lista los cursos. Si el usuario es estudiante, solo devuelve los cursos en los que está inscrito; si es profesor, los que imparte; si es admin, todos.	Autenticado
GET	/api/courses/:id	Devuelve el detalle de un curso, incluyendo el profesor asignado y la lista de estudiantes inscritos.	Autenticado
POST	/api/courses	Crea un curso nuevo (nombre, código, profesor asignado).	admin
PUT	/api/courses/:id	Actualiza los datos de un curso existente.	admin, profesor (propio)
DELETE	/api/courses/:id	Elimina un curso.	admin

/api/grades — Notas

Método	Ruta	Descripción	Acceso
GET	/api/grades?courseId=	Lista las notas de un curso. Un estudiante solo ve las suyas; un profesor ve las de su curso; un admin ve todas.	Autenticado
POST	/api/grades	Registra una nota nueva para un estudiante en un curso (valor, actividad, comentario opcional).	profesor, admin
PUT	/api/grades/:id	Modifica una nota ya existente.	profesor (propio curso), admin
DELETE	/api/grades/:id	Elimina un registro de nota.	admin

/api/users — Usuarios

Método	Ruta	Descripción	Acceso
GET	/api/users	Lista todos los usuarios registrados, con su rol.	admin

Método	Ruta	Descripción	Acceso
GET	/api/users/:id	Devuelve el detalle de un usuario.	admin, el propio usuario
PUT	/api/users/:id	Actualiza datos de perfil (nombre, correo) o el rol del usuario.	admin, el propio usuario (sin cambiar rol)
DELETE	/api/users/:id	Elimina o desactiva una cuenta de usuario.	admin

/api/posts — Comentarios / avisos

Método	Ruta	Descripción	Acceso
GET	/api/posts?courseId=	Lista los comentarios o avisos publicados en un curso, ordenados por fecha.	Autenticado
POST	/api/posts	Crea un comentario nuevo dirigido a un curso o a un docente en particular.	estudiante, profesor, admin
DELETE	/api/posts/:id	Elimina un comentario (propio o cualquiera, si es admin).	autor del post, admin

```
const router = express.Router();

router.get('/', authMiddleware, roleMiddleware('admin', 'profesor', 'estudiante'), getGrades);
router.get('/course/:courseId', authMiddleware, idParamValidation, validationMiddleware, getGradesByCourse);
router.get('/student/:studentId', authMiddleware, idParamValidation, validationMiddleware, getGradesByStudent);
router.get('/summary/:studentId', authMiddleware, idParamValidation, validationMiddleware, getStudentSummary);
router.get('/:id', authMiddleware, idParamValidation, validationMiddleware, getGradeById);
router.post('/', authMiddleware, roleMiddleware('admin'), gradeValidation, validationMiddleware, createGrade);
router.put('/:id', authMiddleware, roleMiddleware('admin', 'profesor'), idParamValidation, gradeValidation, validationMiddleware, updateGrade);
router.delete('/:id', authMiddleware, roleMiddleware('admin'), idParamValidation, validationMiddleware, deleteGrade);

module.exports = router;
```

Middleware de autenticación y control de roles

El sistema usa dos middlewares que se ejecutan en cadena antes de llegar al controlador: uno verifica que el token JWT sea válido, y el otro revisa si el rol del usuario tiene permiso para esa acción en particular. Esta separación permite reutilizar el mismo middleware de roles en rutas distintas, solo cambiando la lista de roles permitidos.

Middleware de validación de token (auth.middleware.js)

Este middleware lee el token del encabezado Authorization, lo verifica con la clave secreta del servidor y, si es válido, agrega los datos del usuario (id y rol) al objeto req para que los siguientes middlewares y controladores puedan usarlos.

```
función verificarToken(req, res, next):  
  encabezado = req.headers['authorization']  
  
  si encabezado no existe:  
    devolver 401 'No se proporcionó token'  
  
  token = encabezado.reemplazar('Bearer ', '')  
  
  intentar:  
    datos = jwt.verificar(token, CLAVE_SECRETA)  
    req.usuario = { id: datos.id, rol: datos.rol }  
    continuar (next())  
  capturar error:  
    devolver 403 'Token inválido o expirado'
```

```
backend > src > middlewares > JS auth.middleware.js > ...  
1  const jwt = require('jsonwebtoken');  
2  const env = require('...../config/env');  
3  const { User } = require('.../models');  
4  
5  module.exports = async (req, res, next) => {  
6    try {  
7      const authHeader = req.headers.authorization;  
8  
9      if (!authHeader || !authHeader.startsWith('Bearer ')) {  
10       | return res.status(401).json({ ok: false, message: 'Token no proporcionado.' });  
11     }  
12  
13     const token = authHeader.replace('Bearer ', '');  
14     const decoded = jwt.verify(token, env.jwt.secret);  
15     const user = await User.findByPk(decoded.sub);  
16  
17     if (!user) {  
18       | return res.status(401).json({ ok: false, message: 'Usuario no válido.' });  
19     }  
20  
21     req.user = {  
22       id: user.id,  
23       email: user.email,  
24       rol: user.rol,  
25     };  
26  
27     return next();  
28   } catch (error) {  
29     | return res.status(401).json({ ok: false, message: 'Token inválido o expirado.' });  
30   }  
31 };  
32
```

Middleware de control de roles (role.middleware.js)

Este middleware se declara como una función que recibe la lista de roles permitidos y devuelve otro middleware. Así, en cada ruta se puede escribir algo como `verificarRol(['admin', 'profesor'])` y el middleware sabe exactamente a quién dejar pasar. Es lo que se conoce como control matricial: cada ruta define su propia combinación de roles permitidos, en vez de tener un único nivel de permisos fijo.

```
función verificarRol(rolesPermitidos):
  devolver función(req, res, next):
    rolUsuario = req.usuario.rol // viene del middleware anterior

    si rolesPermitidos.incluye(rolUsuario):
      continuar (next())
    si no:
      devolver 403 'No tienes permiso para esta acción'

// Ejemplo de uso en una ruta:
router.post('/courses', verificarToken, verificarRol(['admin']), crearCurso)
router.post('/grades', verificarToken, verificarRol(['admin', 'profesor']), crearNota)
```

```
backend > src > middlewares > JS role.middleware.js > <unknown>
1  module.exports = (...allowedRoles) => {
2    return (req, res, next) => {
3      if (!req.user) {
4        return res.status(401).json({
5          ok: false,
6          message: 'Debe iniciar sesión para continuar.',
7        });
8      }
9
10     if (!allowedRoles.includes(req.user.rol)) {
11       return res.status(403).json({
12         ok: false,
13         message: 'No tienes permisos para esta acción.',
14       });
15     }
16
17     return next();
18   };
19   };
```

Matriz de permisos por rol

Recurso / acción	admin	profesor	estudiante
Crear curso	Sí	No	No
Editar curso propio	Sí	Sí	No
Ver notas propias	Sí	Sí (de su curso)	Sí
Registrar/editar notas	Sí	Sí (su curso)	No
Publicar comentarios	Sí	Sí	Sí
Gestionar usuarios	Sí	No	No

Modelos relacionales (Sequelize)

La base de datos se modela con cuatro entidades principales. Sequelize define cada una como una clase de JavaScript (un "modelo"), y las relaciones entre ellas se declaran explícitamente para que el ORM sepa cómo generar los JOIN necesarios.

User

Representa a cualquier persona que usa el sistema, sin importar su rol. El campo rol distingue si es admin, profesor o estudiante, y es el mismo dato que después revisa el middleware de control de roles.

Campo	Tipo	Notas
id	INTEGER (PK)	Autoincremental
nombre	STRING	Requerido
correo	STRING	Único, requerido
contraseña	STRING	Guardada con hash (bcrypt), nunca en texto plano
rol	ENUM('admin','profesor','estudiante')	Define los permisos del middleware de roles

Course

Representa un curso. Se relaciona con User de dos formas distintas: un profesor asignado (belongsTo) y varios estudiantes inscritos (relación muchos a muchos, normalmente mediante una tabla intermedia de inscripciones).

Campo	Tipo	Notas
id	INTEGER (PK)	Autoincremental
nombre	STRING	Nombre del curso
codigo	STRING	Código único del curso
profesorId	INTEGER (FK → User)	Relación belongsTo con User

Grade (Notas)

Cada registro de Grade representa una nota puntual de un estudiante en un curso, asociada a una actividad. Se relaciona con User (estudiante) y con Course.

Campo	Tipo	Notas
id	INTEGER (PK)	Autoincremental
estudianteld	INTEGER (FK → User)	Relación belongsTo con User
cursoId	INTEGER (FK → Course)	Relación belongsTo con Course
actividad	STRING	Nombre de la actividad evaluada
valor	FLOAT	Nota obtenida
comentario	TEXT	Opcional, observación del profesor

Post (Comentarios / avisos)

Representa un mensaje o comentario dirigido a un curso o a un docente en particular. Se relaciona con User (autor) y opcionalmente con Course.

Campo	Tipo	Notas
id	INTEGER (PK)	Autoincremental
autorId	INTEGER (FK → User)	Relación belongsTo con User
cursoId	INTEGER (FK → Course, nullable)	Puede ser nulo si el mensaje es directo a un docente
contenido	TEXT	Texto del comentario
fecha	DATE	Fecha de creación (createdAt de Sequelize)

Diagrama de relaciones

```
User (1) — (N) Course      [profesorId]
User (N) — (N) Course      [inscripciones, tabla intermedia]
User (1) — (N) Grade       [estudianteId]
Course (1) — (N) Grade      [cursoId]
User (1) — (N) Post        [autorId]
Course (1) — (N) Post      [cursoId, nullable]
```

```
1  const sequelize = require('../config/database');
2  const User = require('./user.model')(sequelize);
3  const Course = require('./course.model')(sequelize);
4  const Grade = require('./grade.model')(sequelize);
5  const Post = require('./post.model')(sequelize);
6
7  Course.hasMany(Grade, { foreignKey: 'courseId', as: 'grades' });
8  Grade.belongsTo(Course, { foreignKey: 'courseId', as: 'course' });
9
10 User.hasMany(Grade, { foreignKey: 'studentId', as: 'grades' });
11 Grade.belongsTo(User, { foreignKey: 'studentId', as: 'student' });
12
13 User.hasMany(Post, { foreignKey: 'autorId', as: 'posts' });
14 Post.belongsTo(User, { foreignKey: 'autorId', as: 'autor' });
15 User.hasMany(Post, { foreignKey: 'destinatarioId', as: 'receivedPosts' });
16 Post.belongsTo(User, { foreignKey: 'destinatarioId', as: 'destinatario' });
17 Course.hasMany(Post, { foreignKey: 'courseId', as: 'posts' });
18 Post.belongsTo(Course, { foreignKey: 'courseId', as: 'course' });
19
20 module.exports = { sequelize, User, Course, Grade, Post };
```

Consideraciones finales

Con esto queda cubierta la parte técnica del sistema: cómo están organizadas las tres capas, qué hace cada endpoint de la API, cómo se protege el acceso con JWT y control de roles, y cómo se relacionan los modelos de datos entre sí. En conjunto, esta información debería ser suficiente para que alguien ajeno al desarrollo original pueda entender el funcionamiento interno del sistema sin tener que leer todo el código desde cero.